

Checkpoint 4

- **¿Cuál es la diferencia entre una lista y una tupla en Python?**

Tanto una lista como una tupla sirven para almacenar una serie o colección de datos. Sin embargo, cuentan con unas características diferentes. Para comenzar, la sintaxis es distinta en ambos, una lista se delimita usando [], mientras que una tupla se delimita mediante (). Aún así, la principal diferencia es la mutabilidad, esto es, en el caso de las listas, se pueden modificar los datos, mientras que en una tupla no. Para poner un ejemplo:

```
una_lista = [1,2,3]
una_lista[0] = 4
```

Esto modificará la lista y nos lo devolverá como [4,2,3].

```
una_tupla = (1,2,3)
una_tupla[0] = 4
```

Esto nos devolverá un error ya que las tuplas son inmutables.

- **¿Cuál es el orden de las operaciones?**

En Python, para la prioridad de las operaciones se respeta el principio PEMDAS. Esto es, primero se respetan las parentesis, después van los exponentes, después se realizan las multiplicaciones o divisiones y finalmente las sumas y restas. Por ejemplo, si yo realizo la siguiente operación: $2 + 3 * 4$, la respuesta será 14, ya que se prioriza el $3 * 4$ primero.

- **¿Qué es un diccionario Python?**

Un diccionario Python también es una estructura para almacenar datos, solo que esta vez vienen en pares de "key" o clave y valor. Funciona de la misma forma que en un diccionario de una lengua. En ese caso, buscaríamos una palabra, la cual sería la key o clave, y esta tendría asociada una definición o información, lo cual sería el valor. En un diccionario Python funciona igual, siguiendo la siguiente estructura:

```
un_diccionario = {
    'edad' : 25
    'nombre' : 'Jordan'
    'trabajo' : 'profesor'
}
```

Si pedimos buscar el valor de una clave, nos lo dará

`print (un_diccionario['edad'])`, nos devolverá 25. Los diccionarios Python son mutables. Sirven para organizar mejor los datos, representándolos de una forma más clara y estructurada y ayudándonos a acceder a ellos más rápidamente.

- **¿Cuál es la diferencia entre el método ordenado y la función de ordenación?**

El método ordenado, `.sort()`, ordena los datos de la lista directamente, modificándola, mientras que la función de ordenación, `sorted()`, devuelve una nueva lista ordenada sin modificar la original. Por ejemplo:

Usando método ordenado:

```
una_lista = [3,1,2]
una_lista.sort()
```

```
print(una_lista)
```

El resultado sería: `[1,2,3]`

Usando la función de ordenación:

```
una_lista = [3,1,2]
nueva_lista = sorted(una_lista)
```

```
print(una_lista)
print(nueva_lista)
```

Los resultados serían: `[3,1,2]` y `[1,2,3]`.

- **¿Qué es un operador de reasignación?**

Un operador de reasignación sirve para actualizar una variable. El operador coge el valor de la variable, realiza la operación y le asigna el nuevo valor. Los operadores de reasignación que podemos encontrar son:

- `+=` → suma y asigna
- `-=` → resta y asigna
- `*=` → multiplica y asigna
- `/=` → divide y asigna

Por ejemplo:

```
edad = 25
```

```
edad +=5
```

```
print(edad)
```

El resultado sería 30.

Gracias a estos operadores se puede hacer el código más simple, se mejora la legibilidad y evita repetir la variable.